

Today, we start from taking a look at a mixed integer linear programming problem programmed in Julia, as for the computation of α_i , local indicator for a FDIA. We've seen the equivalent derivation of the problem given by the use of boolean indicators δ .

Continuous variables typically model modern physical phenomena (measurements, physical variables, velocity, currents, etc.).

Boolean variables are typically introduced to model logical conditions, decisions, logical constraints such as "if then", "if and only if", ...

In our case, the logical condition we introduced was $\delta_j = 1 \iff a_j \neq 0$.

Typically, CPS must obey to:

- Physical laws $\rightarrow$ generating continuous variables.
- Rules, logical conditions introduced by humans. A typical case is given by safety conditions: If we're in an industrial plant, a typical condition might be that, if the presence of a human is detected, we could stop robots.
We can translate these logical conditions into mathematical constraints which are computer-readable by introducing boolean variables and associated constraints.

The combination of these elements take the name of MIP.

To make the code in Julia, we use JuMP. It's what we called a modelling language, which is a library that allows writing an optimization problem in machine readable format. After we've written the optimization problem, we need to solve it, and to do this we use solvers packages, given by Gurobi.JL, which is the Julia library used to connect to the Gurobi solver, which is a separate program with respect to Julia. Another choice of solver is given by CBC, with the connector known as CbC.JL.

Topology Attacks

In the grid we have nodes and electrical lines. Not all nodes are always connected, as we have "red boxes", which can be used to connect and disconnect a line. For example, at a given time of the day, the switch could be open.

At a given time, only a set of lines are actually connected, while other might remain closed.

With interrupters, we can change the topology of the grid (set of connected lines), which is time varying.

We call $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ the $\mathcal{G}$, with $\mathcal{V}$ the set of busses, and $\mathcal{E}$ the set of connected cables.

Topology attacks are FDIA aimed at ...

Imagine the operator is inside the Control Center, and cannot see if a breaker is open or closed physically. He needs to rely on sensors. The switches themselves will periodically send a status report to the control center. We denote this as "digital measurement", as it's basically just a 0/1 (telling either close or open).

Power meters, in the other hand, will report analog measurements.

By processing the status report, the operator will compute the current topology (most likely one).

$s \longrightarrow \hat{\mathcal{G}}$ current topology estimated by operator (topology estimation algorithms) based on status report from interrupters ("digital measurements")

When we talk about $\mathcal{G}$, we are referring to the real topology, and $\mathcal{E}$ are the lines which are actually connected. $\hat{\mathcal{G}}$, in the other hand, is the estimated topology.

We thus have:

$$\begin{aligned} z_a &= z + a && \text{attack on analog measurements} \\ s_a &= s + b && \text{attack on digital measurements} \end{aligned}$$

What is the problem? If we change the status report to zero, we have that if we have a sensors sending data for a line that should be closed, we do not have inconsistency anymore.

Topology attacks are attacks on the power measurements and on the status report from the interrupters, conceived in such a way that the operator will see a topology different from the real one. Let's call $\bar{\mathcal{G}}$ the target topology, with $\bar{\mathcal{G}} = \{\mathcal{V}, \bar{\mathcal{E}}\}$. If the attack is successful, then $\hat{\mathcal{G}} = \bar{\mathcal{G}}$.

For example, if we take $\mathcal{E} \setminus \bar{\mathcal{E}}$ what do we get? We get the set of lines which are actually connected in reality, but appears disconnected from the control center. These are called "cyber tripped/open" lines.

With $\mathcal{G}$, we have lines that are not connected, but appear connected.

Undetectability Condition

The other day we've seen the measurement model $z = Hx + e$. We saw that H actually depends also on the topology of the grid. The other day we've assumed the topology is fixed, but we can now write the equation as $z = H(\mathcal{G})x + \mathcal{E}$, stressing that it depends on the topology.

What was the undetectability condition for the simplest FDIA? It was $a = Hc$. If we put this inside the measurements equation (ignoring noises) $z_a = Hx + a$, we have that $= Hx + Hc = H(x + c)$. We're basically saying that $z_a = z + a \in \text{Im}(H)$. We have that $\text{Im}(A) = \{x : Hc, \forall c\}$, so the set we get by multiplying the matrix for any possible vector.

By writing $a = Hc$, we're saying that $a \in \text{Im}(H)$. This means the attack is consistent with the model of the system.

If we take the grid and assume the state to be c , what we read from the sensor is $z = Hc$.

We have that $z_a = z + a \in \text{Im}(H)$ must be consistent with the model of the grid.

Now, in topology attack, the perceived topology is no longer $\mathcal{G}$ but $\bar{\mathcal{G}}$. Then we should have that $z_a = z + a \in \text{Im}(\bar{H})$, as the operator will check consistency of measurements assuming $\bar{H}$ (and not real H).

The attack problem is thus to find a such that $z + a = \bar{H}\hat{x}_a$. So, $z + a$ must be in the image of $\bar{H}$, and $\hat{x}_a$ will be the explanation of the operator, being the estimated state under attack.

We can have $\hat{x} = \underset{x}{\text{argmin}}(z - \bar{H}x)^T W(z - \bar{H}x)$.

We receive data from the field, z (analog data) and s (digital data). We then have a topology process, trying to determine the actual topology. The SE will then run the computation of $\hat{x}$ using the estimated topology, so:

$$\underset{x}{\text{argmin}}(z - H(\hat{\mathcal{G}})x)^T W(z - H(\hat{\mathcal{G}})x)$$

We then have the usual BDD with a correction procedure.

Now, with the attack, the attacker corrupts both z and s in a consistent way, having compatible data. So, if $a + z \in \text{Im}(\bar{H})$, what we get from the SE module is x_a and $\bar{\mathcal{G}}$.